

LINUX LAB ASSIGNMENT

Aditi Biju

261100690027

Part A — Basic grep

Exercise 1 — Search File Contents

Using payload.sh:

Find the line containing wget.

Find the line containing nc.

Find the line containing chmod.

Search for WGET ignoring case.

Display the line numbers containing /tmp.

the number of lines containing /tmp.

Display only the matched word wget.

Search for nc as a whole word.

```
(aduuu@kali)~[/grep_lab]
$ grep "wget" payload.sh
wget http://example.com/file.txt -O /tmp/file.txt

(aduuu@kali)~[/grep_lab]
$ grep "nc" payload.sh
nc -l 4444

(aduuu@kali)~[/grep_lab]
$ grep "chmod" payload.sh
chmod 777 /tmp/file.txt

(aduuu@kali)~[/grep_lab]
$ grep -i "WGET" payload.sh
wget http://example.com/file.txt -O /tmp/file.txt

(aduuu@kali)~[/grep_lab]
$ grep -n "/tmp" payload.sh
2:wget http://example.com/file.txt -O /tmp/file.txt
4:chmod 777 /tmp/file.txt

(aduuu@kali)~[/grep_lab]
$ grep -c "/tmp" payload.sh
2

(aduuu@kali)~[/grep_lab]
$ grep -o "wget" payload.sh
wget

(aduuu@kali)~[/grep_lab]
$ grep -w "nc" payload.sh
nc -l 4444
```

Exercise 2 — Compare Files

Search for wget in:

payload.sh

update.sh

cleanup.sh

Answer:

Which file contains wget?

Which files do not contain wget?

Repeat the search for /tmp.

Which files contain /tmp?

Use grep with multiple filenames

```
(aduuu@kali)~[/grep_lab]
$ nano update.sh

(aduuu@kali)~[/grep_lab]
$ nano cleanup.sh

(aduuu@kali)~[/grep_lab]
$ grep "wget" payload.sh update.sh cleanup.sh
payload.sh:wget http://example.com/file.txt -O /tmp/file.txt

(aduuu@kali)~[/grep_lab]
$ grep -l "wget" payload.sh update.sh cleanup.sh
payload.sh

(aduuu@kali)~[/grep_lab]
$ grep -L "wget" payload.sh update.sh cleanup.sh
update.sh
cleanup.sh

(aduuu@kali)~[/grep_lab]
$ grep "/tmp" payload.sh update.sh cleanup.sh
payload.sh:wget http://example.com/file.txt -O /tmp/file.txt
payload.sh:chmod 777 /tmp/file.txt

(aduuu@kali)~[/grep_lab]
$ grep -l "/tmp" payload.sh update.sh cleanup.sh
payload.sh

(aduuu@kali)~[/grep_lab]
$ grep -c "/tmp" payload.sh update.sh cleanup.sh
payload.sh:2
update.sh:0
cleanup.sh:0
```

Exercise 3 — Inverse Search

Using system.log:

Display all lines that do not contain INFO.

Display all lines that do not contain ERROR.

Display all lines except those containing WARNING.

Count the number of non-INFO entries.

```
(aduuu@kali)~[/grep_lab]
$ nano system.log

(aduuu@kali)~[/grep_lab]
$ grep -v "INFO" system.log
2026-08-30 10:02:20 WARNING Failed login attempt detected
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:05:30 ERROR FAILED authentication check
2026-08-30 10:06:45 WARNING Suspicious activity detected

(aduuu@kali)~[/grep_lab]
$ grep -v "ERROR" system.log
2026-08-30 10:00:01 INFO System started successfully
2026-08-30 10:01:15 INFO User login successful
2026-08-30 10:02:20 WARNING Failed login attempt detected
2026-08-30 10:04:10 INFO Database service restarted
2026-08-30 10:06:45 WARNING Suspicious activity detected

(aduuu@kali)~[/grep_lab]
$ grep -v "WARNING" system.log
2026-08-30 10:00:01 INFO System started successfully
2026-08-30 10:01:15 INFO User login successful
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:04:10 INFO Database service restarted
2026-08-30 10:05:30 ERROR FAILED authentication check

(aduuu@kali)~[/grep_lab]
$ grep -vc "INFO" system.log
4
```

Part B — Security Log Analysis

Exercise 4 — Analyze system.log

Find all ERROR entries:

Find all WARNING entries:

Find the FAILED entry:

Count the number of ERROR entries:

Display line numbers containing WARNING:

Search for error irrespective of case:

Determine whether the word Malware occurs without displaying output:

```
(aduuu@kali)~[/grep_lab]
$ grep "ERROR" system.log
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:05:30 ERROR FAILED authentication check

(aduuu@kali)~[/grep_lab]
$ grep "WARNING" system.log
2026-08-30 10:02:20 WARNING Failed login attempt detected
2026-08-30 10:06:45 WARNING Suspicious activity detected

(aduuu@kali)~[/grep_lab]
$ grep "FAILED" system.log
2026-08-30 10:05:30 ERROR FAILED authentication check

(aduuu@kali)~[/grep_lab]
$ grep -c "ERROR" system.log
2

(aduuu@kali)~[/grep_lab]
$ grep -n "WARNING" system.log
3:2026-08-30 10:02:20 WARNING Failed login attempt detected
7:2026-08-30 10:06:45 WARNING Suspicious activity detected

(aduuu@kali)~[/grep_lab]
$ grep -i "error" system.log
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:05:30 ERROR FAILED authentication check

(aduuu@kali)~[/grep_lab]
$ grep -q "Malware" system.log

(aduuu@kali)~[/grep_lab]
$ echo $?
1
```

Exercise 5 — Context Investigation

An ERROR event has been detected in system.log.

Display two lines after each ERROR.

Display two lines before each ERROR.

Display one line before and one line after each ERROR.

Display two lines around FAILED.

Why can examining events before and after an error be useful during incident investigation?

```

(aduuu@kali)~/grep_lab
$ grep -A 2 "ERROR" system.log
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:04:10 INFO Database service restarted
2026-08-30 10:05:30 ERROR FAILED authentication check
2026-08-30 10:06:45 WARNING Suspicious activity detected

(aduuu@kali)~/grep_lab
$ grep -B 2 "ERROR" system.log
2026-08-30 10:01:15 INFO User login successful
2026-08-30 10:02:20 WARNING Failed login attempt detected
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:04:10 INFO Database service restarted
2026-08-30 10:05:30 ERROR FAILED authentication check

(aduuu@kali)~/grep_lab
$ grep -C 1 "ERROR" system.log
2026-08-30 10:02:20 WARNING Failed login attempt detected
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:04:10 INFO Database service restarted
2026-08-30 10:05:30 ERROR FAILED authentication check
2026-08-30 10:06:45 WARNING Suspicious activity detected

(aduuu@kali)~/grep_lab
$ grep -C 2 "FAILED" system.log
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:04:10 INFO Database service restarted
2026-08-30 10:05:30 ERROR FAILED authentication check
2026-08-30 10:06:45 WARNING Suspicious activity detected

```

Examining the events before and after an error helps identify what caused the error and what happened after it. It gives a better understanding of the sequence of events during an incident.

Part C — egrep / grep -E

Exercise 6 — Search Multiple Security Events

Using system.log, write a single command to find lines containing:

ERROR

WARNING

FAILED

```

(aduuu@kali)~/grep_lab
$ grep -E "ERROR|WARNING|FAILED" system.log
2026-08-30 10:02:20 WARNING Failed login attempt detected
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:05:30 ERROR FAILED authentication check
2026-08-30 10:06:45 WARNING Suspicious activity detected

```

Exercise 7 — Multiple Suspicious Commands

Using payload.sh, search for any of:

wget

curl

nc

chmod using a single extended regular expression.

Answer:

Which patterns were found?

wget, nc, chmod

Which pattern was not found?

curl

How many lines contain at least one of these patterns?

```
(aduuu@kali)~/grep_lab
$ grep -E "wget|curl|nc|chmod" payload.sh
wget http://example.com/file.txt -O /tmp/file.txt
nc -l 4444
chmod 777 /tmp/file.txt

(aduuu@kali)~/grep_lab
$ grep -Ec "wget|curl|nc|chmod" payload.sh
3
```

Exercise 8 — Search Multiple Files

Which files contain at least one selected command?

Display only the filenames containing matches.

Display matching lines with line numbers.

Count the matching lines in each file.

```
(aduuu@kali)~/grep_lab
$ grep -El "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
payload.sh

(aduuu@kali)~/grep_lab
$ grep -El "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
payload.sh

(aduuu@kali)~/grep_lab
$ grep -En "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
payload.sh:2:wget http://example.com/file.txt -O /tmp/file.txt
payload.sh:3:nc -l 4444
payload.sh:4:chmod 777 /tmp/file.txt

(aduuu@kali)~/grep_lab
$ grep -Ec "wget|curl|nc|chmod" payload.sh update.sh cleanup.sh
payload.sh:3
update.sh:0
cleanup.sh:0
```

Part D — patterns.txt

Your dataset already contains patterns.txt.

Exercise 9 — Pattern File Investigation

Use patterns.txt to:

Search payload.sh.

Search system.log.

Search config.conf.

Search all .sh files.

Display only filenames containing at least one pattern from patterns.txt.

Display matching lines without printing filenames.

Which files contain indicators listed in patterns.txt?

```
(aduuu@kali)~/grep_lab
$ grep -f patterns.txt payload.sh
wget http://example.com/file.txt -O /tmp/file.txt
nc -l 4444
chmod 777 /tmp/file.txt

(aduuu@kali)~/grep_lab
$ grep -f patterns.txt system.log
2026-08-30 10:02:20 WARNING Failed login attempt detected
2026-08-30 10:03:05 ERROR Unable to connect to database
2026-08-30 10:05:30 ERROR FAILED authentication check
2026-08-30 10:06:45 WARNING Suspicious activity detected

(aduuu@kali)~/grep_lab
$ grep -f patterns.txt config.conf
grep: config.conf: No such file or directory

(aduuu@kali)~/grep_lab
$ nano config.conf

(aduuu@kali)~/grep_lab
$ grep -f patterns.txt config.conf
username=admin
password=admin123

(aduuu@kali)~/grep_lab
$ grep -f patterns.txt *.sh
payload.sh:wget http://example.com/file.txt -O /tmp/file.txt
payload.sh:nc -l 4444
payload.sh:chmod 777 /tmp/file.txt

(aduuu@kali)~/grep_lab
$ grep -lf patterns.txt *.sh
payload.sh

(aduuu@kali)~/grep_lab
$ grep -hf patterns.txt *.sh
wget http://example.com/file.txt -O /tmp/file.txt
nc -l 4444
chmod 777 /tmp/file.txt
```

Part E — Regex Using filelist.txt

Exercise 10 — Beginning and End of Line

Using filelist.txt: Find filenames beginning with malware.

Find filenames beginning with script.

Find filenames ending in .sh.

Find filenames ending in .py.

Find filenames ending in .log.

Find filenames beginning with p and ending in .sh

```

(aduuu@kali)~[/grep_lab]
$ grep "^malware" filelist.txt
malware1.sh
malware2.sh
malwareA.sh

(aduuu@kali)~[/grep_lab]
$ grep "^script" filelist.txt
script1.py
script2.py

(aduuu@kali)~[/grep_lab]
$ grep "\.sh$" filelist.txt
malware1.sh
malware2.sh
malwareA.sh
payload.sh
update.sh
cleanup.sh

(aduuu@kali)~[/grep_lab]
$ grep "\.py$" filelist.txt
script1.py
script2.py
scanner.py

(aduuu@kali)~[/grep_lab]
$ grep "\.log$" filelist.txt
access.log
system.log

(aduuu@kali)~[/grep_lab]
$ grep "^p.*\.sh$" filelist.txt
payload.sh

```

Exercise 11 — Character Classes

Using filelist.txt:

Find malware1.sh and malware2.sh using one pattern.

Find filenames containing a digit.

Find filenames containing digits 1 or 2.

Find filenames beginning with m or p.

Find filenames beginning with s or u.

```

(aduuu@kali)~[/grep_lab]
$ grep "malware[12]\.sh$" filelist.txt
malware1.sh
malware2.sh

(aduuu@kali)~[/grep_lab]
$ grep "[0-9]" filelist.txt
malware1.sh
malware2.sh
script1.py
script2.py
user1.txt

(aduuu@kali)~[/grep_lab]
$ grep "[12]" filelist.txt
malware1.sh
malware2.sh
script1.py
script2.py
user1.txt

(aduuu@kali)~[/grep_lab]
$ grep "^[mp]" filelist.txt
malware1.sh
malware2.sh
malwareA.sh
payload.sh
password.txt

(aduuu@kali)~[/grep_lab]
$ grep "^[su]" filelist.txt
script1.py
script2.py
scanner.py
update.sh
system.log

```


Exercise 12 — Negated Character Classes

Using filelist.txt:

Find filenames that do not begin with m.

Find filenames that do not begin with m or p.

Find malware filenames where the character following malware is not a digit.



```
(aduuu@kali)~/grep_lab
$ grep "^[^m]" filelist.txt
script1.py
script2.py
canner.py
payload.sh
update.sh
cleanup.sh
report.txt
access.log
system.log
password.txt
ser1.txt

(aduuu@kali)~/grep_lab
$ grep "^[^mp]" filelist.txt
script1.py
script2.py
canner.py
update.sh
cleanup.sh
report.txt
access.log
system.log
ser1.txt

(aduuu@kali)~/grep_lab
$ grep "^malware[^0-9]" filelist.txt
malwareA.sh
```

Exercise 13 —

Write a shell script that accepts a filename and determines whether it exists.

Test with:

payload.sh

report.txt

unknown.sh

Write a script that accepts a path and determines whether it is:

a regular file,

a directory,

or neither.

Test with:

payload.sh

system.log

quarantine

backup

Unknown

```
(aduuu@kali)-[~/grep_lab]
$ nano check_file.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x check_file.sh

(aduuu@kali)-[~/grep_lab]
$ ./check_file.sh
Enter filename: payload.sh
File exists
```

```
(aduuu@kali)-[~/grep_lab]
$ chmod +x check_file.sh

(aduuu@kali)-[~/grep_lab]
$ ./check_file.sh
Enter filename: payload.sh
File exists

(aduuu@kali)-[~/grep_lab]
$ ./check_file.sh
Enter filename: report.txt
File does not exist

(aduuu@kali)-[~/grep_lab]
$ ./check_file.sh
Enter filename: unknown.sh
File does not exist

(aduuu@kali)-[~/grep_lab]
$ █
```

```
(aduuu@kali)-[~/grep_lab]
$ nano check_path.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x check_path.sh

(aduuu@kali)-[~/grep_lab]
$ ./check_path.sh
Enter path: system.log
Regular file
```

```
(aduuu@kali)-[~/grep_lab]
$ ./check_path.sh
Enter path: system.log
Regular file

(aduuu@kali)-[~/grep_lab]
$ ./check_path.sh
Enter path: quarantine
Neither

(aduuu@kali)-[~/grep_lab]
$ ./check_path.sh
Enter path: backup
Neither

(aduuu@kali)-[~/grep_lab]
$ ./check_path.sh
Enter path: unknown
Neither
```

Exercise 15

Write a script that reports whether a selected file is:

readable,

writable,

executable.

Test with:

payload.sh

update.sh

report.txt

config.conf

Expected style of output:

File: payload.sh

Readable : Yes

Writable : Yes

Executable : Yes

```
(aduuu@kali)~[/grep_lab]
$ nano file_permissions.sh

(aduuu@kali)~[/grep_lab]
$ chmod +x file_permissions.sh

(aduuu@kali)~[/grep_lab]
$ ./file_permissions.sh
Enter filename: payload.sh

File: payload.sh

Readable   : Yes
Writable   : Yes
Executable : Yes

(aduuu@kali)~[/grep_lab]
$ ./file_permissions.sh
Enter filename: update.sh

File: update.sh

Readable   : Yes
Writable   : Yes
Executable : No

(aduuu@kali)~[/grep_lab]
$ ./file_permissions.sh
Enter filename: report.txt

File: report.txt

Readable   : No
Writable   : No
Executable : No

(aduuu@kali)~[/grep_lab]
$ ./file_permissions.sh
Enter filename: config.conf

File: config.conf

Readable   : Yes
Writable   : Yes
Executable : No
```

Exercise 16 —

Compare:

payload.sh

empty.txt

Write a script that determines whether each file contains data.

Display:

File contains data

or:

File is empty

```
(aduuu@kali)-[~/grep_lab]
$ nano file_data.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x file_data.sh

(aduuu@kali)-[~/grep_lab]
$ ./file_data.sh
Enter filename: payload.sh
File contains data

(aduuu@kali)-[~/grep_lab]
$ ./file_data.sh
Enter filename: empty.txt
File is empty
```

Part G

Exercise 17 — File Validation Using []

Write a script that accepts a filename and checks:

Does it exist?

Is it a regular file?

Is it readable?

Is it non-empty?

```
(aduuu@kali)-[~/grep_lab]
$ nano validate.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x validate.sh

(aduuu@kali)-[~/grep_lab]
$ ./validate.sh
Enter filename: payload.sh
File exists
Regular file: Yes
Readable: Yes
Non-empty: Yes
```

Exercise 18 — [] + grep

Write a script with the following logic:

Does file exist?

↓

Is it a regular file?

↓

Is it readable?

↓

Is it non-empty?

↓

Search for "wget"

If wget is detected, display:

Selected indicator detected – investigate further

Otherwise:

Selected indicator not detected

Test using both:

payload.sh

update.sh

```
(aduuu@kali)-[~/grep_lab]
$ nano validation.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x validation.sh

(aduuu@kali)-[~/grep_lab]
$ ./validation.sh
Enter filename: payload.sh
Selected indicator detected – investigate further

(aduuu@kali)-[~/grep_lab]
$ ./validation.sh
Enter filename: update.sh
Selected indicator not detected
```

Part H

Exercise 19 — Repeat File Tests Using [[]]

Repeat Exercise 17 using:

```
(aduuu@kali)-[~/grep_lab]
$ nano repeat.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x repeat.sh

(aduuu@kali)-[~/grep_lab]
$ ./repeat.sh
Enter filename: payload.sh
File exists
Regular file: Yes
Readable: Yes
Non-empty: Yes
```

Exercise 20 — Regex Using [[]]

Write a shell script that accepts a filename and checks whether the filename ends with .sh.

Use:

```
[[ "$filename" =~ regex ]]
```

Display:

Shell script identified for inspection

or:

File does not have a .sh extension

Test:

payload.sh

scanner.py

report.txt

```
(aduuu@kali)-[~/grep_lab]
$ nano regex.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x regex.sh

(aduuu@kali)-[~/grep_lab]
$ ./regex.sh
Enter filename: payload.sh
Shell script identified for inspection

(aduuu@kali)-[~/grep_lab]
$ ./regex.sh
Enter filename: scanner.py
File does not have a .sh extension

(aduuu@kali)-[~/grep_lab]
$ ./regex.sh
Enter filename: report.txt
File does not have a .sh extension
```

Exercise 21 — Multiple Extensions Using [[]]

Extend the previous script to identify:

.sh

.py

.pl

files.

Test:

payload.sh

scanner.py

report.txt

config.conf

Do not classify these as malicious. Display:

Script file – inspect if required

```
(aduuu@kali)-[~/grep_lab]
$ nano q21.sh

(aduuu@kali)-[~/grep_lab]
$ chmod +x q21.sh

(aduuu@kali)-[~/grep_lab]
$ ./q21.sh
Enter filename: payload.sh
Script file – inspect if required

(aduuu@kali)-[~/grep_lab]
$ ./q21.sh
Enter filename: scanner.py
Script file – inspect if required

(aduuu@kali)-[~/grep_lab]
$ ./q21.sh
Enter filename: report.txt
Not a script file

(aduuu@kali)-[~/grep_lab]
$ ./q21.sh
Enter filename: config.conf
Not a script file
```

Part I — grep + File Operators + Constructs

Exercise 22 — Executable File Content Analysis

Write a script that:

Accepts a filename.

Checks whether it exists.

Checks whether it is a regular file.

Checks whether it is executable.

If executable, searches for wget

Test with:

payload.sh

update.sh

report.txt

```
(aduuu@kali) - [~/grep_lab]
$ nano execute.sh

(aduuu@kali) - [~/grep_lab]
$ chmod +x execute.sh

(aduuu@kali) - [~/grep_lab]
$ ./execute.sh
Enter filename: payload.sh
File is executable
wget found

(aduuu@kali) - [~/grep_lab]
$ ./execute.sh
Enter filename: update.sh
File is not executable

(aduuu@kali) - [~/grep_lab]
$ ./execute.sh
Enter filename: report.txt
File does not exist
```

Exercise 23 — egrep + File Operators

Write a script that performs content analysis only when the selected file is:

a regular file,

readable,

non-empty.

If the checks succeed, use egrep/grep -E to search for:

wget|curl|nc|chmod

Display:

One or more selected indicators detected

if a match occurs.

Otherwise:

Selected indicators not detected


```
(aduuu@kali)~[/grep_lab]
$ nano reg.sh

(aduuu@kali)~[/grep_lab]
$ chmod +x reg.sh

(aduuu@kali)~[/grep_lab]
$ ./reg.sh
Enter filename: payload.sh
Regular file: Yes
Readable: Yes
Non-empty: Yes
One or more selected indicators detected

(aduuu@kali)~[/grep_lab]
$ ./reg.sh
Enter filename: update.sh
Regular file: Yes
Readable: Yes
Non-empty: Yes
Selected indicators not detected
```

Part J — Configuration File Investigation

Exercise 24 — Sensitive Information Search

Use config.conf.

First verify that it:

exists,

is a regular file,

is readable,

is non-empty.

Then use egrep to search for:

password|username

Display the matching lines.

```
(aduuu@kali)~[/grep_lab]
$ nano config.conf

(aduuu@kali)~[/grep_lab]
$ egrep "password|username" config.conf
username=admin
password=admin123
```

Configuration files need extra protection because they can contain usernames and passwords. If someone gets the file, they could use those details to access the system.

Part K

Exercise 25 — Suspicious File Triage

Write a Bash program called:

`file_check.sh`

The program should ask the user to enter a filename.

It should:

Determine whether the path exists.

Determine whether it is a regular file.

Determine whether it is a directory.

If it is a regular file, determine whether it is readable.

Determine whether it is writable.

Determine whether it is executable.

Determine whether it is empty/non-empty.

If readable and non-empty, use `grep` to search for `wget`.

Use `egrep/grep -E` to search for `wget|curl|nc|chmod`.

Use `$?` to determine whether a selected search succeeded.

Use `[[ ]]` with regex to determine whether the filename ends in `.sh`.

Display a summary of the findings.

Example:

Enter filename: `payload.sh`

----- FILE INVESTIGATION -----

Exists : Yes

Regular File : Yes

Directory : No

Readable : Yes

Writable : Yes

Executable : Yes

Non-empty : Yes

Shell Script : Yes

```
(aduuu@kali)-[~/grep_lab]  
$ nano file_check.sh
```

```
(aduuu@kali)-[~/grep_lab]  
$ chmod +x file_check.sh
```

```
(aduuu@kali)-[~/grep_lab]  
$ ./file_check.sh  
Enter filename: payload.sh
```

```
—— FILE INVESTIGATION ——
```

```
Exists      : Yes  
Regular File : Yes  
Directory   : No  
Readable    : Yes  
Writable    : Yes  
Executable  : Yes  
Non-empty   : Yes  
Shell Script : Yes
```

```
wget        : Found  
Selected indicators : Found
```

```
Further inspection recommended
```